just to use a null object, and in fact RSpec provides a nice spy method for this purpose:

```
>> mario = spy('Mario')
=> #<Double "Mario">
>> Game.play(mario)
=> #<Double "Mario">
>> expect(mario).to have_received(:jump)
=> nil
```

Not only does this alias save you a bit of code, it also expresses your intent better. You're declaring up front that you're going to use this test double as a spy.

Origins: Pure, Partial, and Verifying Doubles

Now that we've seen the different usage modes of test doubles, let's look at where they come from.

Pure Doubles

All of the test doubles you've written so far in this chapter are *pure doubles*: they're purpose-built by rspec-mocks and consist entirely of behavior you add to them. You can pass them into your project code just as if they were the real thing.

Pure doubles are flexible and easy to get started with. They're best for testing code where you can pass in dependencies. Unfortunately, real-world projects are not always so tester-friendly, and you'll need to turn to more powerful techniques.

Partial Doubles

Sometimes, the code you're testing doesn't give you an easy way to inject dependencies. A hard-coded class name may be lurking three layers deep in that API method you're calling. For instance, a lot of Ruby projects call Time.now without providing a way to override this behavior during testing.

To test these kinds of codebases, you can use a *partial double*. These add mocking and stubbing behavior to existing Ruby objects. That means any object in your system can be a partial double. All you have to do is expect or allow a specific message, just like you'd do for a pure double:

```
>> random = Random.new
=> #<Random:0x007ff2389554e8>
>> allow(random).to receive(:rand).and_return(0.1234)
=> #<RSpec::Mocks::MessageExpectation #<Random:0x007ff2389554e8>.rand(any ↩
arguments)>
>> random.rand
=> 0.1234
```